

JAVA PROGRAMMING

Chapter 3: Data Types in Java

By Akanshu Jamwal

Table of Contents

1. What Are Data Types?
2. Java is Statically Typed
3. The 8 Primitive Data Types
4. Primitive Type Details
5. Primitive vs Reference Types
6. String (Reference Type)
7. Why Data Types Matter for Placements
8. Quick Comparison Table
9. Practical Exercise
10. What's Next?

1. What Are Data Types?

A data type specifies the type of data that a variable can hold. It tells Java what kind of values can be stored in a variable and how much memory to allocate.

Example:

```
int dailyPracticeMinutes = 90;
```

dailyPracticeMinutes	→ variable
int	→ data type
90	→ value stored

This means the variable can store only **integer values**. If you try to store a decimal number, the compiler will reject it.

2. Java is Statically Typed

Java requires every variable to be declared with a data type before it is used. The compiler must know the data type in advance.

✓ **This is valid:**

```
double mockAccuracy = 84.5;
```

■ **This is invalid:**

```
mockAccuracy = 84.5;    // No data type
```

3. The 8 Primitive Data Types

Java has 8 predefined primitive data types. These are the most basic data storage types available in Java.

4. Primitive Type Details

1■■■ boolean

Purpose: Stores logical values: true or false

Default value: false

Used in: Conditions, validation checks, access control logic

```
boolean hasActiveSubscription = true;  
System.out.println(hasActiveSubscription);
```

2■■■ byte

Range: -128 to 127

Size: 8-bit

Use Case: When memory efficiency matters

```
byte moduleLevel = 5;  
System.out.println(moduleLevel);  
// Default value: 0
```

3■■■ short

Range: -32,768 to 32,767

Size: 16-bit

Use Case: When integer range is small but larger than byte

```
short contestScore = 1500;  
System.out.println(contestScore);  
// Default value: 0
```

4■■■ int

Range: -2,147,483,648 to 2,147,483,647

Size: 32-bit

Use Case: Most commonly used integer type for general purposes

```
int totalUsers = 250000;  
System.out.println(totalUsers);  
// Default value: 0
```

■ **Tip for Placements:** For most competitive programming and placement problems, **int is the default integer choice**. Use it unless the problem requires larger numbers.

5■■■ long

Range: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

Size: 64-bit

Use Case: Used for very large numbers

```
long platformVisits = 150000000000L;  
System.out.println(platformVisits);  
// Notice the L at the end
```

Important: Without the L suffix, Java treats large numbers as int, which causes overflow errors.

6■■■ double

Size: 64-bit floating-point type

Precision: Approx 15-17 significant digits

Use Case: For decimal/fractional numbers

```
double averageCompletionTime = 3.75;  
System.out.println(averageCompletionTime);  
// Default value: 0.0
```

■■ **Warning:** Never use double for precise currency calculations due to floating-point rounding issues. Use BigDecimal instead for financial applications.

7■■ float

Size: 32-bit floating-point type

Precision: Approx 6-7 significant digits (less precise than double)

Use Case: When memory is limited or precision requirements are lower

```
float discountRate = 12.5F;  
System.out.println(discountRate);  
// Notice the F suffix (mandatory)
```

Without the F suffix, Java treats decimal values as double by default.

8■■■ char

Stores: A single Unicode character

Size: 16-bit

Range: Unicode values from 0 to 65,535

```
char gradeLevel = 'A';
System.out.println(gradeLevel);

// You can also assign using Unicode:
char symbol = '\u2605'; // ★ star symbol
System.out.println(symbol);
// Default value: '\u0000'
```

5. Primitive vs Reference Types

The 8 types above are **primitive types**. Java also supports **reference types**:

- String
- Arrays
- Classes
- Objects

Primitive Types	Reference Types
Store actual values	Store memory addresses (references)
Fixed memory size	Variable memory size
Faster access	Slightly slower access
Examples: int, double, boolean	Examples: String, Array, Class

6. String (Reference Type)

Strings are not primitive. They are objects of `java.lang.String`. Strings are sequences of characters and are very commonly used in Java programs.

```
String trackName = "DSA Mastery Track";  
System.out.println(trackName);  
  
String welcomeMessage = "Welcome to CodeHelp!";  
System.out.println(welcomeMessage);
```

7. Why Data Types Matter for Placements

In interviews, companies test:

- Proper data type selection
- Overflow handling
- Precision issues
- Memory optimization
- Type mismatch errors
- Edge case handling with large numbers

Example Problem:

Problem: Given N students with test scores up to 10^{12} , find the maximum score.

Wrong Choice: Using int will cause integer overflow!

Correct Choice: Use long to handle values up to 9.2×10^{18}

Strong data type understanding prevents:

- Runtime errors and crashes
- Wrong answers due to overflow
- Edge case failures
- Performance issues
- Memory wastage

8. Quick Comparison Table

Type	Size	Range/Purpose	Default
boolean	1 byte	true/false	false
byte	1 byte	-128 to 127	0
short	2 bytes	-32,768 to 32,767	0
int	4 bytes	-2^{31} to $2^{31}-1$	0
long	8 bytes	-2^{63} to $2^{63}-1$	0L
float	4 bytes	~6-7 digits precision	0.0f
double	8 bytes	~15-17 digits precision	0.0
char	2 bytes	Unicode 0 to 65,535	'\u0000'

9. Practical Exercise

Try creating variables for:

- Number of contests attended (use int)
- Average problem solving time (use double)
- Whether premium member or not (use boolean)
- Grade in mock test (use char)
- Total platform visits (use long)

Run them in your IDE and observe:

- Type behavior
- Literal assignment rules
- Output differences
- Default values

Consistent hands-on practice builds real clarity.

10. What's Next?

Now that you understand data types, the next logical steps in your Java learning journey are:

- Type Casting (converting between data types)
- Operators (arithmetic, logical, comparison)
- Taking Input using Scanner class
- Conditional Statements (if-else, switch)
- Loops (for, while, do-while)

■ **Key Takeaway:**

Strong foundations in data types make DSA and backend development much easier. Understanding when to use which type can mean the difference between correct and incorrect solutions!